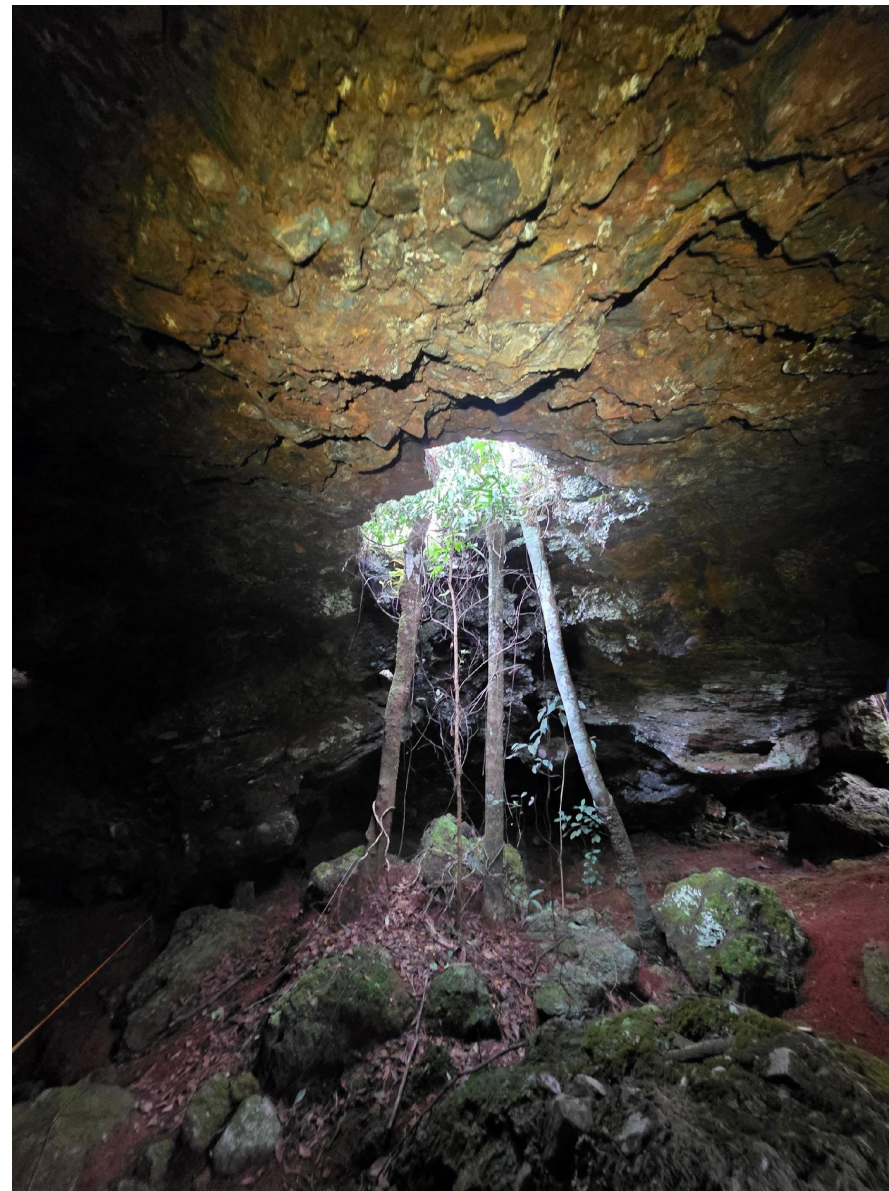


Programação e Desenvolvimento de Software I

Passagem de parâmetros

Prof. Héctor Azpúrua
(slides adaptados do Prof. Pedro Olmo)

Inspeção



Passagem de parâmetros

O que é?

- Toda função define um processamento a ser realizado

```
void funcao(int a) {  
    printf("a: %d\n", a);  
}
```

- Este processamento depende dos **valores dos parâmetros** da função
- Assim, para usar uma função, um programa precisa fornecer a ela os parâmetros adequados
- Exemplos:
 - Para calcular o seno de 30° , escrevemos: `sin(PI/6);`
 - Para calcular o valor absoluto de $a-b$, escrevemos: `abs(a-b);`
 - Para calcular o mdc de 12 e 8, escrevemos: `mcd(12, 8);`

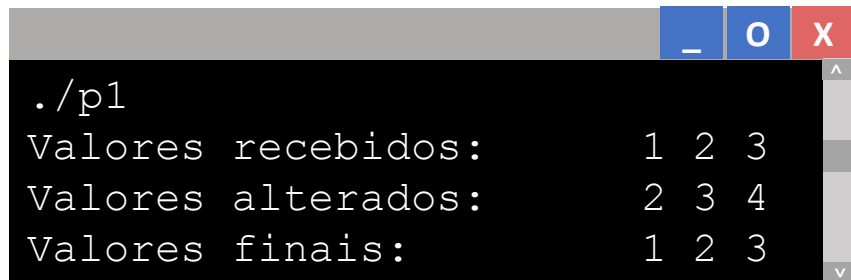
Passagem de parâmetros

Mecanismos

- O mecanismo de informar os valores a serem processados pela função chama-se passagem de parâmetros
- A Linguagem C define duas categorias de passagem de parâmetros:
 - Passagem por **valor** e
 - Passagem por **endereço** (ou **passagem por referência**)
- Normalmente, a passagem de parâmetros a uma função é **por valor**
- Mas, como os parâmetros de uma função são variáveis **locais**, alguns aspectos devem ser observados...

Passagem por valor

- O que este programa vai exibir?
 - Os valores ficam iguais no main!



A terminal window with a dark background and light text. The window title bar shows a minus sign, a maximize button, and a close button. The output of the program is as follows:

```
./p1
Valores recebidos:      1  2  3
Valores alterados:      2  3  4
Valores finais:         1  2  3
```

```
void alterar(int x, int y, int z) {
    printf("Valores recebidos:\t%d %d %d\n", x, y, z);
    x++;
    y++;
    z++;
    printf("Valores alterados:\t%d %d %d\n", x, y, z);
}
```

```
int main() {
    int a = 1;
    int b = 2;
    int c = 3;
    alterar(a, b, c);
    printf("Valores finais:\t\t%d %d %d\n", a, b, c);

    return 0;
}
```

Passagem por valor

Mecanismos

- Os valores das variáveis `a`, `b` e `c` não foram modificados na função **`alterar`**
 - Por quê?
- O tipo de passagem de parâmetros utilizado é **por valor**
 - Ou seja, são feitas apenas **cópias** dos valores das variáveis `a`, `b`, e `c` nas variáveis `x`, `y` e `z`

Escopo: Função `main`

Variável	Valor	Endereço
<code>a</code>	1	0xF1
<code>b</code>	2	0xF2
<code>c</code>	3	0xF3

Escopo: Função `alterar`

<code>x++</code>	Variável	Valor	Endereço
	<code>x</code>	1	0xA5
<code>y++</code>	<code>y</code>	2	0xA6
<code>z++</code>	<code>z</code>	3	0xA7

Passagem por valor

Mecanismos

- Os valores das variáveis `a`, `b` e `c` não foram modificados na função **alterar**
 - Por quê?
- O tipo de passagem de parâmetros utilizado é **por valor**
 - Ou seja, são feitas apenas **cópias** dos valores das variáveis `a`, `b`, e `c` nas variáveis `x`, `y` e `z`

Escopo: Função `main`

Variável	Valor	Endereço
a	1	0xF1
b	2	0xF2
c	3	0xF3

Apenas os conteúdos de `x`, `y` e `z` são alterados

Escopo: Função `alterar`

	Variável	Valor	Endereço
x++	x	2	0xA5
y++	y	3	0xA6
z++	z	4	0xA7

Passagem por referência

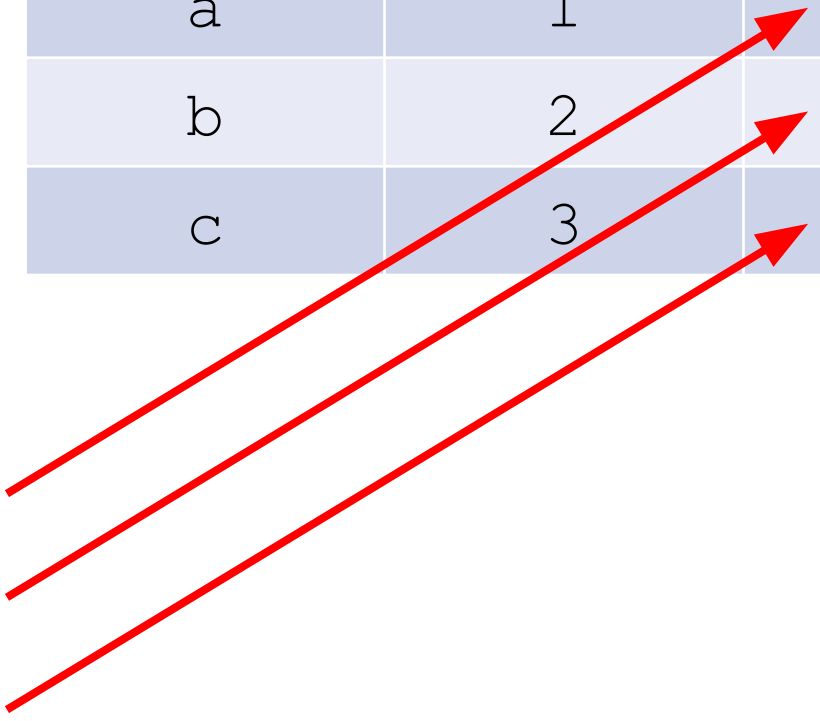
Mecanismos

- Mas, e se quisermos que a função **modifique os valores** das variáveis `a`, `b` e `c` passadas a ela como parâmetros?
- Neste caso, em vez de passar para a função os valores destas variáveis, é preciso passar os seus endereços!
 - Como assim?
- Considere, por exemplo, que as variáveis `a`, `b` e `c` correspondem, respectivamente, aos endereços (hexadecimais) `0xF1`, `0xF2` e `0xF3`

Passagem por referência

Mecanismos

Variável	Valor	Endereço
a	1	0xF1
b	2	0xF2
c	3	0xF3



- Sabemos que:

- $\&a = 0xF1$

- $\&b = 0xF2$

- $\&c = 0xF3$

- $a = 1, b = 2, c = 3$

Passagem por referência

Mecanismos

Variável	Valor	Endereço
a	1	0xF1
b	2	0xF2
c	3	0xF3

- Considere uma variável declarada como:

`int *x;`

- `x` é um ponteiro para `int`, ou seja, `x` é uma variável que armazena o endereço de uma variável do tipo `int`
- Considere agora:

`x = &a;`

- Neste caso, `x` armazena qual valor?
 - 0xF1

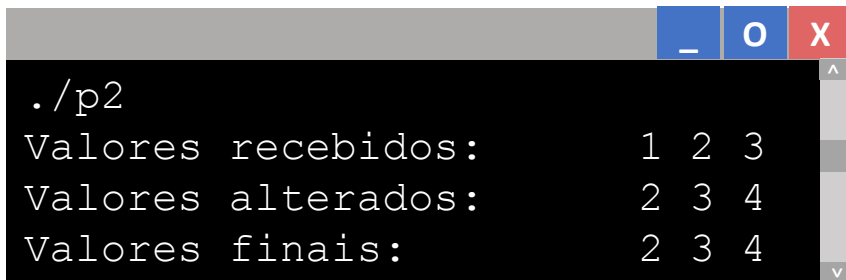
Define-se `*x`, como sendo o valor contido na posição de memória apontada por `x`. Ou seja, `*x` vale 1

Passagem por referência

- O que este programa vai exibir?

```
void alterar(int* x, int* y, int* z) {  
    printf("Valores recebidos:%d %d %d\n", *x, *y, *z);  
    (*x)++;  
    (*y)++;  
    (*z)++;  
    printf("Valores alterados:%d %d %d\n", *x, *y, *z);  
}
```

```
int main() {  
    int a = 1;  
    int b = 2;  
    int c = 3;  
  
    alterar(&a, &b, &c);  
    printf("Valores finais:%d %d %d\n", a, b, c);  
  
    return 0;  
}
```



A terminal window with a dark background and a title bar containing minus, maximize, and close buttons. The output of the program is displayed as follows:

```
./p2  
Valores recebidos:      1  2  3  
Valores alterados:      2  3  4  
Valores finais:         2  3  4
```

Passagem por referência

Mecanismos

- Os valores das variáveis `a`, `b` e `c` foram SIM modificados na função **alterar**
 - Por quê?
- O tipo de passagem de parâmetros utilizado é **por referência**
 - Ou seja, são passados os endereços das variáveis `a`, `b`, e `c` para os ponteiros `x`, `y` e `z`

Altera o conteúdo de `a`, `b` e `c`!

Escopo: Função `main`

Variável	Valor	Endereço
<code>a</code>	1	0xF1
<code>b</code>	2	0xF2
<code>c</code>	3	0xF3

`(*x)++ == a++`

`(*y)++ == b++`

`(*z)++ == c++`

Escopo: Função `alterar`

Variável	Valor	Endereço
<code>x</code>	0xF1	0xA5
<code>y</code>	0xF2	0xA6
<code>z</code>	0xF3	0xA7

Passagem por referência

Atenção

- Considere que o endereço de `x` é `0xFFF1`:

```
int x = 1;
```

```
int *a;
```

```
a = &x;
```

- Neste caso:

```
a = 0xFFF1; // endereço de x
```

```
*a = 1; // pois *a = x = *(&x) = 1
```

- Logo:

```
&>(*a) = &x = 0xFFF1 = a
```

- Portanto:

```
&>(*a) == a
```


Passagem de parâmetros

Problema I

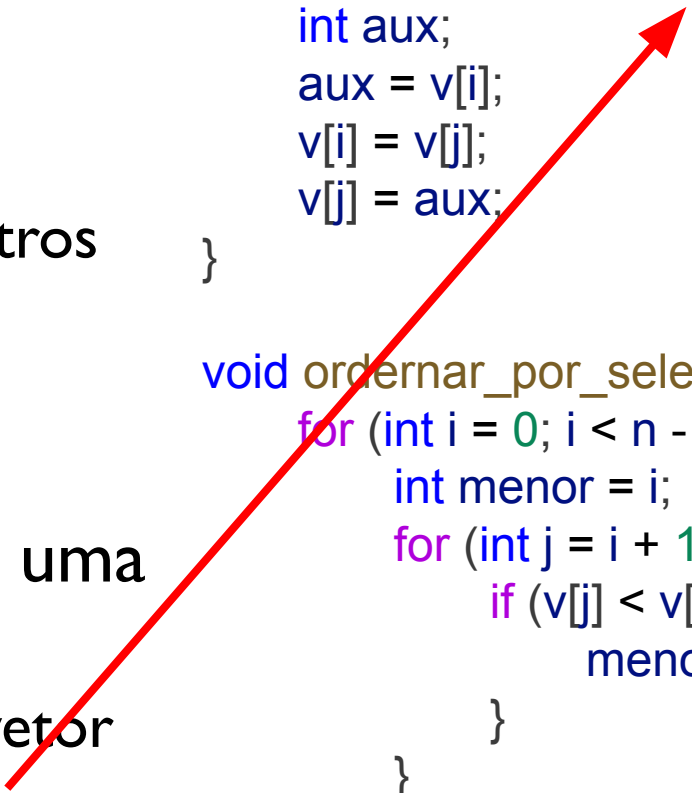
- Implemente uma função que classifique os elementos de um vetor em ordem crescente usando o algoritmo de **ordenação por seleção**
- Teste a função com **dados aleatórios**

Passagem de parâmetros

Ordenação por seleção

- Esse código funciona?
 - Para haver a troca, os parâmetros não devem ser passados por referência (**endereço**)?
- Observe como um vetor é passado como parâmetro de uma função
 - O número de elementos do vetor **não precisa ser declarado**

```
void troca(int v[], int i, int j) {  
    int aux;  
    aux = v[i];  
    v[i] = v[j];  
    v[j] = aux;  
}  
  
void ordenar_por_selecao(int v[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int menor = i;  
        for (int j = i + 1; j < n; j++) {  
            if (v[j] < v[menor]) {  
                menor = j;  
            }  
        }  
        troca(v, i, menor);  
    }  
}
```



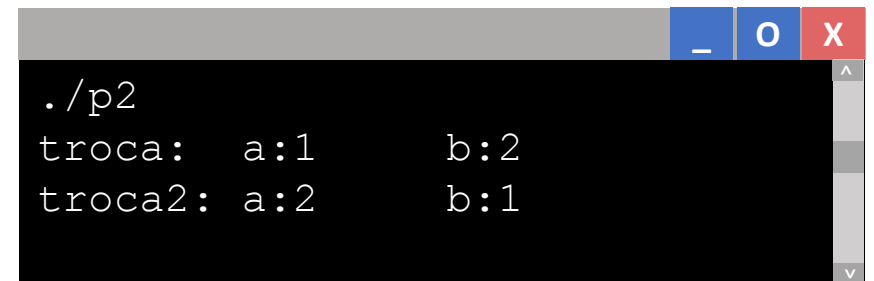
Passagem de parâmetros

Quando usar valor ou referência?

- Quando deve ser feita a passagem de parâmetros por referência?
 - Veja o exemplo:

```
void troca(int a, int b) {  
    int aux;  
    aux = a;  
    a = b;  
    b = aux;  
    printf("%d-%d", a, b);  
}  
  
void troca2(int* a, int* b) {  
    int aux;  
    aux = *a;  
    *a = *b;  
    *b = aux;  
    printf("%d-%d", *a, *b);  
}
```

```
int main() {  
    int a = 1;  
    int b = 2;  
  
    troca(a, b);  
    printf("troca: a:%d\tb:%d\n", a, b);  
  
    troca2(&a, &b);  
    printf("troca2: a:%d\tb:%d\n", a, b);  
  
    return 0;  
}
```

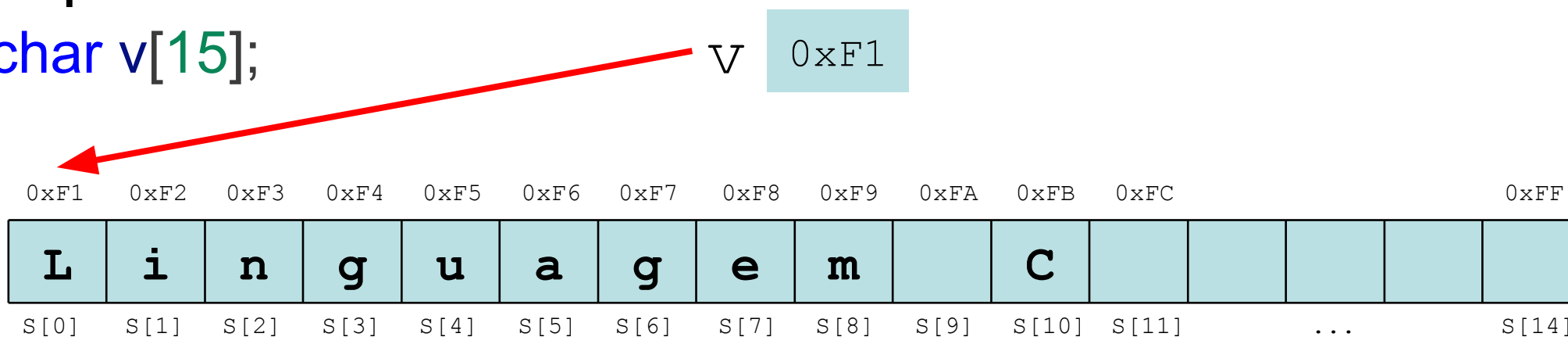


```
./p2  
troca:  a:1      b:2  
troca2: a:2      b:1
```

Vetores como parâmetros

- No caso de **uma função ter como parâmetro um vetor**, temos um caso particular de grande importância
- Por quê? Porque o nome de um vetor nada mais é que um **ponteiro para sua primeira posição**
- Exemplo:

- `char v[15];`



Vetores como parâmetros

Acesso via índices

- Mas, então como é possível usarmos a notação:

```
char nome_do_vetor[] = "hellow";  
nome_do_vetor[indice];
```

- Isto pode ser facilmente explicado, desde que se entenda que a notação acima é absolutamente equivalente a:

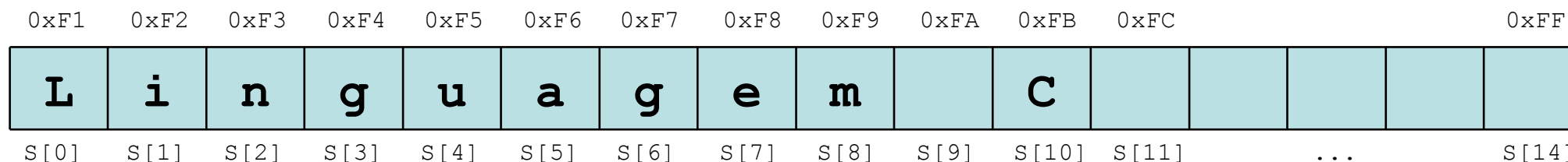
```
*(nome_do_vetor + indice)
```

- Mas, o que significa **somar (ou subtrair) um valor a um ponteiro?**

Vetores como parâmetros

Acesso via índices

- Quando incrementamos um ponteiro, ele passa a **apontar para o próximo valor** do mesmo tipo
- Exemplo:
 - Ao incrementar um ponteiro para `char`, ele anda 1 byte na memória
 - Ao incrementar um ponteiro para `double` ele anda 8 bytes



`v = &v[0]`

`*v = v[0] = 'L'`

`*(v + 3) = v[3] = 'g'`

← São equivalentes!

Vetores como parâmetros

- Logo, como o nome de um **vetor** é também um **ponteiro**, a passagem de parâmetro para vetores é sempre **por referência**
 - Qualquer modificação ocorrida no vetor dentro da função será, na realidade, feita sobre o **parâmetro usado na chamada**

```
void ordenar_por_selecao(int v[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int menor = i;  
        for (int j = i + 1; j < n; j++) {  
            if (v[j] < v[menor]) {  
                menor = j;  
            }  
        }  
        troca(v, i, menor);  
    }  
}
```

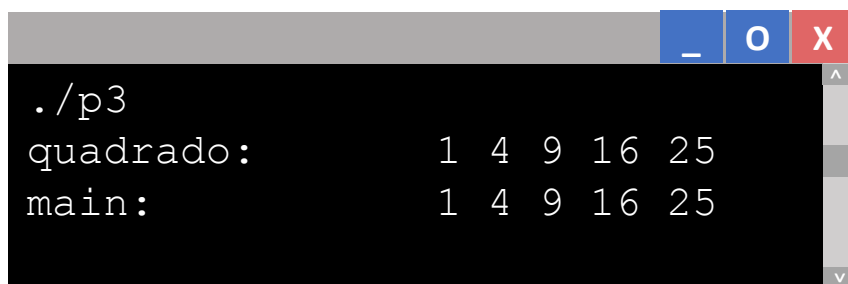
```
int meu_vetor[] = {5, 2, 33, 3, 1};  
ordenar_por_selecao(meu_vetor, 5);
```

Na definição da função, podemos substituir `int x[]` por `int x[5]` ou ainda `int *x`

Alterações no vetor `v` dentro da função `ordenar_por_seleção` serão também realizadas no vetor `meu_vetor`

Vetores como parâmetros

- Os valores serão alterados?
 - Sim!



```
./p3
quadrado:      1  4  9 16 25
main:          1  4  9 16 25
```

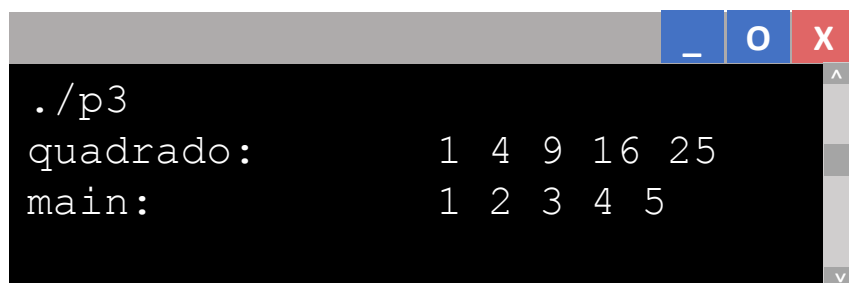
- O que devemos fazer se desejarmos que os elementos de um vetor, passado como parâmetro para uma função, **não sejam alterados**?
- Uma possibilidade...
 - Dentro da função é preciso **atribuir os elementos do vetor a uma variável local**

```
void quadrado(int v[], int n) {
    for (int i = 0; i < n; i++) {
        v[i] = v[i] * v[i];
    }
    printf("quadrado:\t");
    for (int i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("\n");
}
```

```
int main() {
    int a[] = {1, 2, 3, 4, 5};
    int tam = sizeof(a) / sizeof(a[0]);
    quadrado(a, tam);
    printf("main:\t\t");
    for (int i = 0; i < tam; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

Vetores como parâmetros

- Os valores serão alterados?
 - Não! Pois é feita uma copia de `v` em `tmp` ☺



```
./p3
quadrado:      1  4  9 16 25
main:         1  2  3  4  5
```

- Atenção!** Observe que a cópia dos elementos foi feita um a um
 - O que aconteceria se fizéssemos:
`tmp = v`?

```
void quadrado2(int v[], int n) {
    int tmp[n];
    for (int i = 0; i < n; i++)
        tmp[i] = v[i];

    for (int i = 0; i < n; i++)
        tmp[i] = tmp[i] * tmp[i];

    printf("quadrado:\t");
    for (int i = 0; i < n; i++)
        printf("%d ", tmp[i]);

    printf("\n");
}
```

```
int main() {
    int a[] = {1, 2, 3, 4, 5};
    int tam = sizeof(a) / sizeof(a[0]);
    quadrado2(a, tam);
    printf("main:\t\t");
    for (int i = 0; i < tam; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Diferença entre vetores e ponteiros

- Há uma **diferença importante** entre o nome de um vetor e um ponteiro:
 - Um ponteiro é uma variável, mas o **nome de um vetor não é uma variável**
- Isto significa, que não se consegue alterar o endereço que é apontado pelo "nome do vetor"
- Exemplo:

```
int v[10];  
int *p, i;  
p = &i;
```

```
v = v + 2;  
v++;  
v = p;
```

Operações inválidas

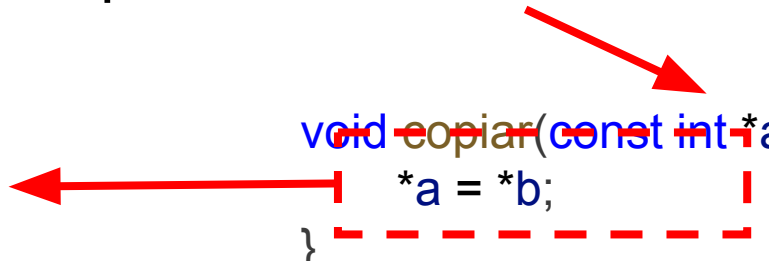
```
p = v;  
p = v + 2;
```

Operações válidas

Qualificador `const`

- Para indicar que um parâmetro de função não deve ser alterado, usa-se o qualificador **`const`**

Se existir na função uma atribuição de valor a um parâmetro declarado como constante, a compilação indicará **um erro**



```
void copiar(const int *a, int *b) {  
    *a = *b;  
}
```

```
int main() {  
    int a = 1;  
    int b = 2;  
    printf("antes: a = %d, b = %d\n", a, b);  
    copiar(&a, &b);  
    printf("depois: a = %d, b = %d\n", a, b);  
    return 0;  
}
```

Matrizes como parâmetros

- Parâmetros do tipo matriz precisam do tamanho declarado
 - Pelo menos da sua **ultima dimensão!**

```
int main() {  
    int arr[3][3] = { {1, 2, 3},  
                     {4, 5, 6},  
                     {7, 8, 9}  
    };  
    print(3, 3, arr);  
    return 0;  
}
```

```
void print(int n, int m, int arr[3][3]) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++)  
            printf("%d ", arr[i][j]);  
        printf("\n");  
    }  
}
```

```
void print(int n, int m, int arr[][3]) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++)  
            printf("%d ", arr[i][j]);  
        printf("\n");  
    }  
}
```

```
void print(int n, int m, int arr[n][m]) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++)  
            printf("%d ", arr[i][j]);  
        printf("\n");  
    }  
}
```



Matrizes como parâmetros

- Parâmetros do tipo matriz precisam do tamanho declarado
 - Pelo menos da sua **ultima dimensão!**

```
int main() {  
    int arr[3][3] = { {1, 2, 3},  
                     {4, 5, 6},  
                     {7, 8, 9}  
    };  
    print(3, 3, arr);  
    return 0;  
}
```

```
void print(int n, int m, int arr[][]) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++)  
            printf("%d ", arr[i][j]);  
        printf("\n");  
    }  
}
```



Perguntas?

- E-mail:
 - hector@dcc.ufmg.br
- Material da disciplina:
 - <https://pedroolmo.github.io/teaching/pdsI.html>
- Github:
 - <https://github.com/h3ct0r>



Héctor Azpúrua
h3ct0r